IronRDP React WebVDI Client:

A Modern WebAssembly-based Remote Desktop Solution

Team 1: Jeongbin Kim / Mentor : 하병현

KAIST School of Computing

Abstract:

To address the lack of modern, maintainable interfaces for client-side VDI, this project develops a robust React-based frontend that effectively operationalizes the IronRDP WebAssembly engine. We architected a strictly typed, modular application using React 19 and Vite 7 that seamlessly orchestrates the complex asynchronous communication between the WASM core and direct WebSocket tunnels. This contribution replaces the legacy Apache Guacamole and Iron Svelte client with a high-performance, responsive solution, proving the viability of secure Zero-Client RDP within a standard modern web ecosystem.

Background

To overcome the scalability limitations of server-side VDI and the maintainability constraints of existing WASM prototypes, this research aims to engineer a **production-grade "Zero-Client" Remote Desktop interface**. The primary objective is to prove that a modern, type-safe web framework (React) can effectively operationalize low-level WebAssembly engines to deliver secure, high-performance RDP sessions without intermediary rendering servers.

Goals

- Modernize the Client Ecosystem
- Operationalize WASM Integration
- Validate Zero-Trust Architecture
- Enhance Usability and Responsiveness

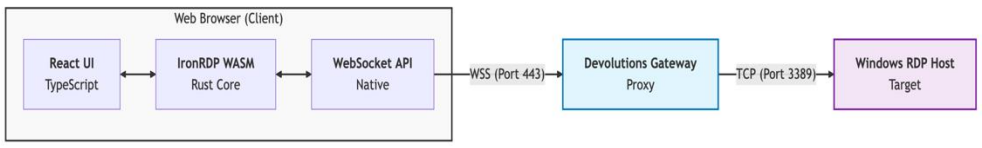
Related Work & Comparative analysis

Solution Category	Representative	Pros	Cons
Commercial Cloud VDI	Microsoft Azure Virtual Desktop	• Native performance & compatibility • Fully managed infrastructure • High reliability SLA	• **High recurring cost** (Per-user pricing) • Vendor lock-in (Proprietary) • Complex enterprise setup required
Server-Side Rendering	Apache Guacamole	• Mature ecosystem (VNC/RDP/SSH) • Clientless (HTML5 only) • Broad browser compatibility	• **Server Bottlenecks**: CPU usage scales linearly • **Latency:** "Decode > Re-encode" pipeline adds lag • **Privacy:** Session data decrypted on server
Early WASM Prototype	IronRDP (Svelte Client)	• Proof of concept for Client-Side RDP • Minimal server footprint • Open Source (Rust/WASM)	• **Maintenance Risk**: Lack of strict Type Safety • **Poor UX:** Non-responsive UI, limited features • **Dev Experience:** Hard to extend/contribute
This Research	IronRDP (React Client)	• **Zero-Server-Load:** 100% Client-Side Processing • **Strict Type Safety:** React 19 + TypeScript • **Modern UX:** Responsive, accessible, modular	• Browser constraints (TCP access requires proxy) • Initial load time (WASM binary size)

Devolutions, "IronRDP: Rust implementation of the Microsoft Remote Desktop Protocol," *GitHub Repository*, 2024. [Online]. Available: github.com/Devolutions/IronRDP.
Apache Software Foundation, "Apache Guacamole: Clientless Remote Desktop Gateway," 2024. [Online]. Available: guacamole.apache.org.
Microsoft Corporation, "[MS-RDPBCGR]: Remote Desktop Protocol: Basic Connectivity and Graphics Remoting Specification," Microsoft Open Specifications, 2024. [Online]. Available: learn.microsoft.com.
A. Haas, et al., "Bringing the web up to speed with WebAssembly," in *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2017, pp. 185–200.

Implementations & Architecture

Component	Technology	Version	Role
Frontend	React	v19.1.1	UI Library
Build Tool	Vite	v7.1.7	Bundler & HMR
Language	TypeScript	v5.9.3	Strict Type Safety
Core Engine	Rust/WASM	wasm-pack	Protocol Handling



Feature Statement

Dynamic Resolution Scaling:

- Implemented a ResizeObserver in React that detects browser window changes.
- Triggers the RDP DisplayControl channel via WASM to auto-negotiate resolution with the Windows host, ensuring the remote desktop always fits the viewport.

Immersive Mode:

- Integrated the HTML5 Fullscreen API with the RDP canvas, allowing users to toggle a native-like remote experience without browser UI distractions.

Reproducible Engineering Standard:

- Authored a rigorous "Zero-to-Hero" deployment guide, standardizing the setup of Rust, WASM, and Node.js environments.
- Documented granular troubleshooting steps for network (NLA) and protocol errors, significantly lowering the barrier to entry for educational and open-source contributors.

Future & Contribution

A. Engineering Rigor for Open Source

To ensure this project serves as a viable candidate for official adoption by the IronRDP community, development adhered to strict software engineering standards rather than ad-hoc prototyping:

- **Architecture:** Decoupled logic (WASM interop) from UI (React components) to facilitate future maintenance and testing.
- **Documentation:** Created comprehensive, reproducibile deployment guides to lower the barrier for future contributors.

B. The Path to Upstreaming

The roadmap focuses on refining the client for open-source submission:

1. **Stabilization:** Final performance profiling and resolution of minor CSS layout shifts during dynamic resizing.
2. **RBI Implementation:** Extending the current VDI foundation to support **RBI**, enabling single-application streaming
3. **Upstream Contribution:** Submitting a formal Pull Request to the IronRDP repository to propose this React client as the new default web interface.